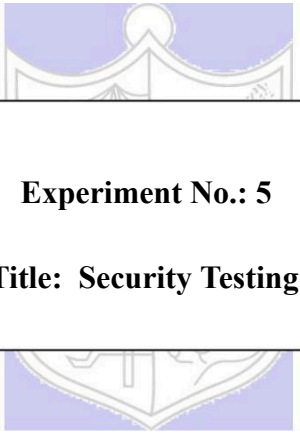


Experiment No.: 5

Title: Security Testing

Batch: B–1 **Roll No.: 16010422234** **Experiment No.: 5**

Aim: To perform Security testing using any open source tool.

Resources needed: Internet, Chrome Browser

Theory:

Security Testing is a type of Software Testing that uncovers vulnerabilities, threats, risks in a software application and prevents malicious attacks from intruders. The purpose of Security Tests is to identify all possible loopholes and weaknesses of the software system which might result in a loss of information, revenue, and reputation at the hands of the employees or outsiders of the Organization.

Why is Security Testing Important?
The main goal of Security Testing is to identify the threats in the system and measure its potential vulnerabilities, so the threats can be encountered and the system does not stop functioning or cannot be exploited. It also helps in detecting all possible security risks in the system and helps developers to fix the problems through coding.

Types of Security Testing in Software Testing
There are seven main types of security testing as per Open Source Security Testing methodology manual. They are explained as follows:

- **Vulnerability Scanning:** This is done through automated software to scan a system against known vulnerability signatures.
- **Security Scanning:** It involves identifying network and system weaknesses, and later provides solutions for reducing these risks. This scanning can be performed for both Manual and Automated scanning.
- **Penetration testing:** This kind of testing simulates an attack from a malicious hacker. This testing involves analysis of a particular system to check for potential vulnerabilities to an external hacking attempt.
- **Risk Assessment:** This testing involves analysis of security risks observed in the organization. Risks are classified as Low, Medium and High. This testing recommends controls and measures to reduce the risk.
- **Security Auditing:** This is an internal inspection of Applications and Operating systems for security flaws. An audit can also be done via line by line inspection of code
- **Ethical hacking:** It's hacking an Organization Software systems. Unlike malicious hackers, who steal for their own gains, the intent is to expose security flaws in the system.

- **Posture Assessment:** This combines Security scanning, Ethical Hacking and Risk Assessments to show an overall security posture of an organization.

Security Testing throughout SDLC

It is always agreed, that cost will be more if we postpone security testing after software implementation phase or after deployment. So, it is necessary to involve security testing in the SDLC life cycle in the earlier phases.

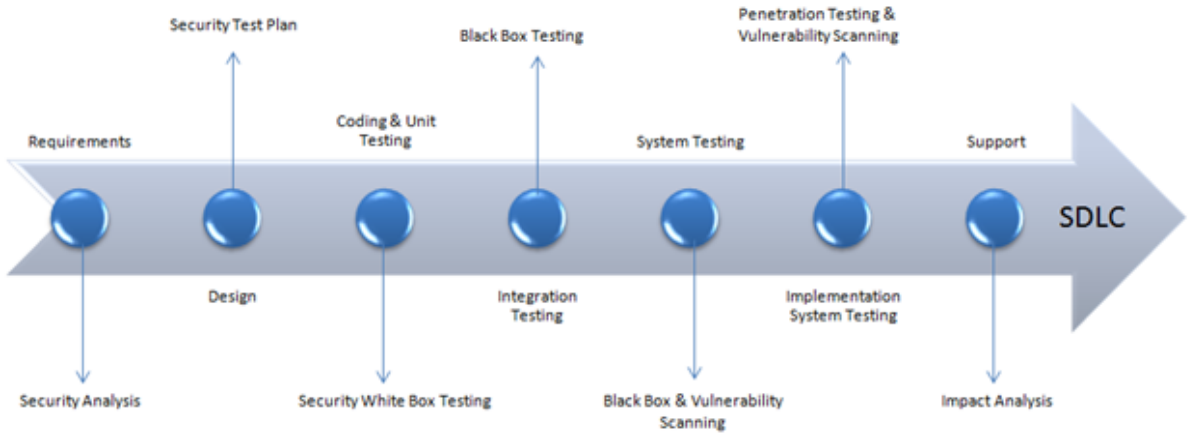


Fig 1: Security processes to be adopted for every phase in SDLC

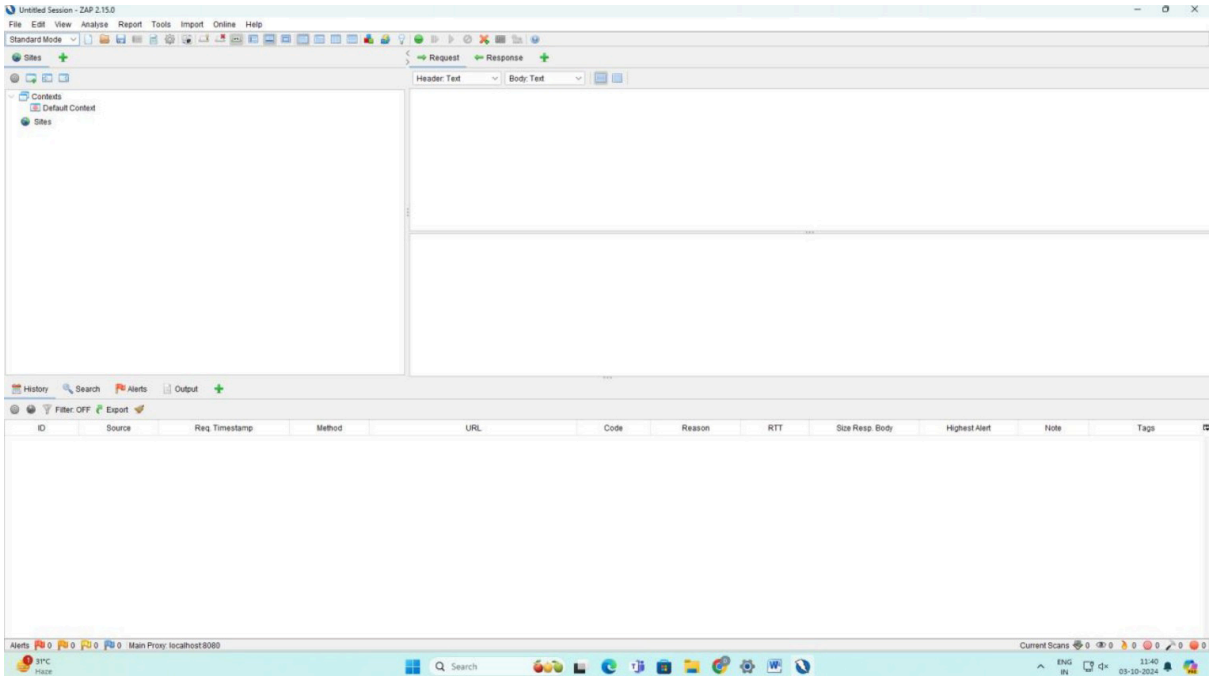
Following are the open source tools which are useful in security testing:

- Zed Attack Proxy (ZAP)
- Wfuzz
- Wapiti
- W3af
- SQLmap
- SonarQuobe
- Nogotofail
- IRONWASP
- Grabber
- Arachni

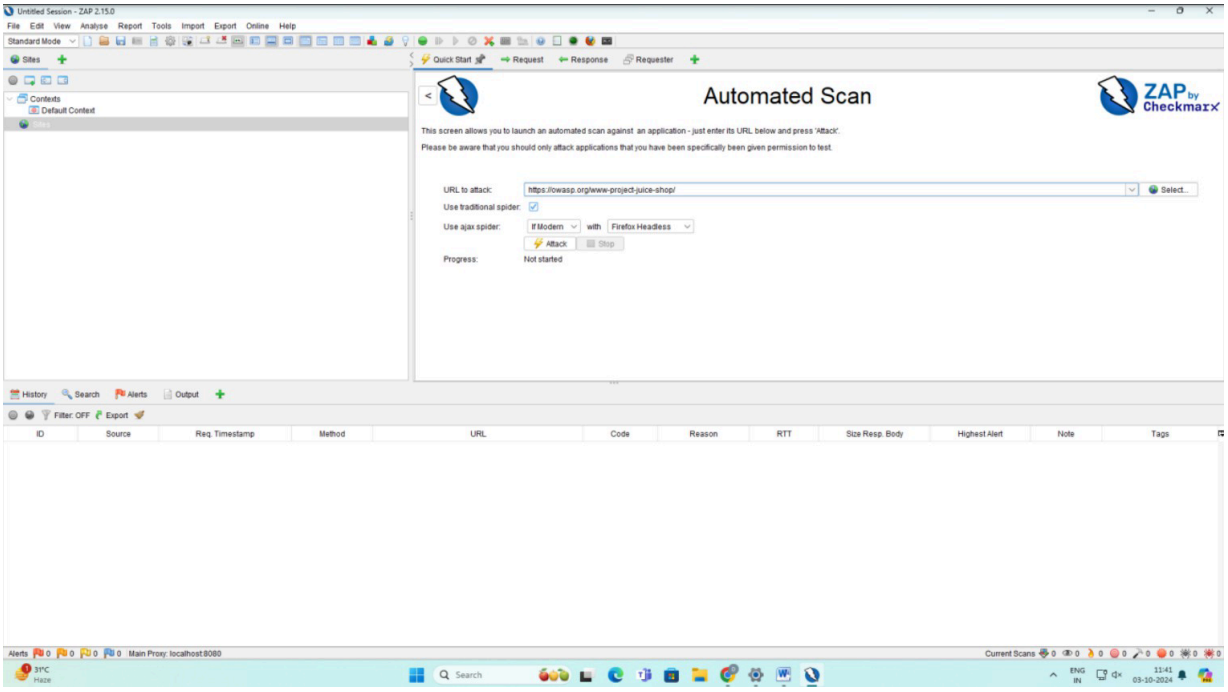
Procedure:

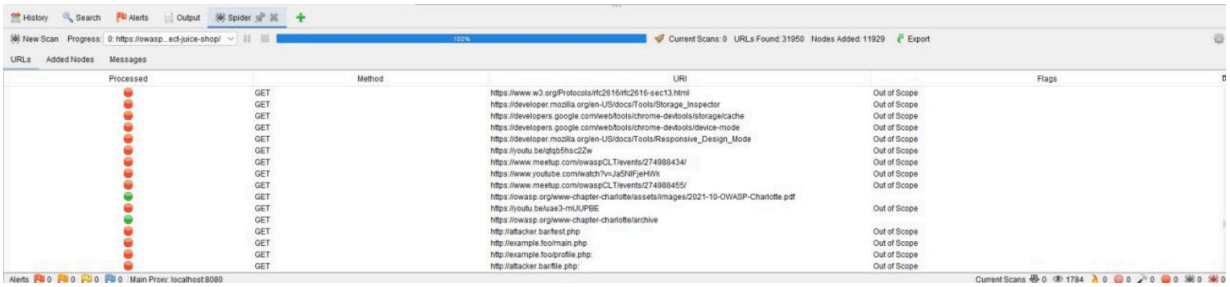
1. Explore any one tool for Security testing.
2. Perform Security testing of any website and generate the report for the same.
3. Analyze and summarize reports for quality and performance of web pages.
4. Suggest improvements based on your analysis.

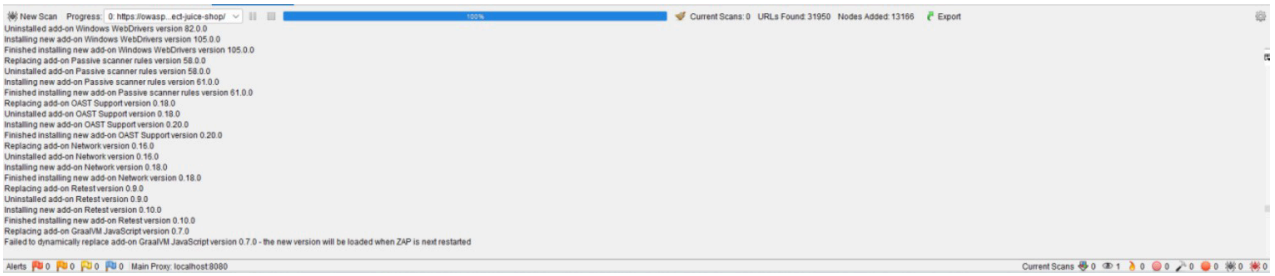
1. Using ZAP Tool

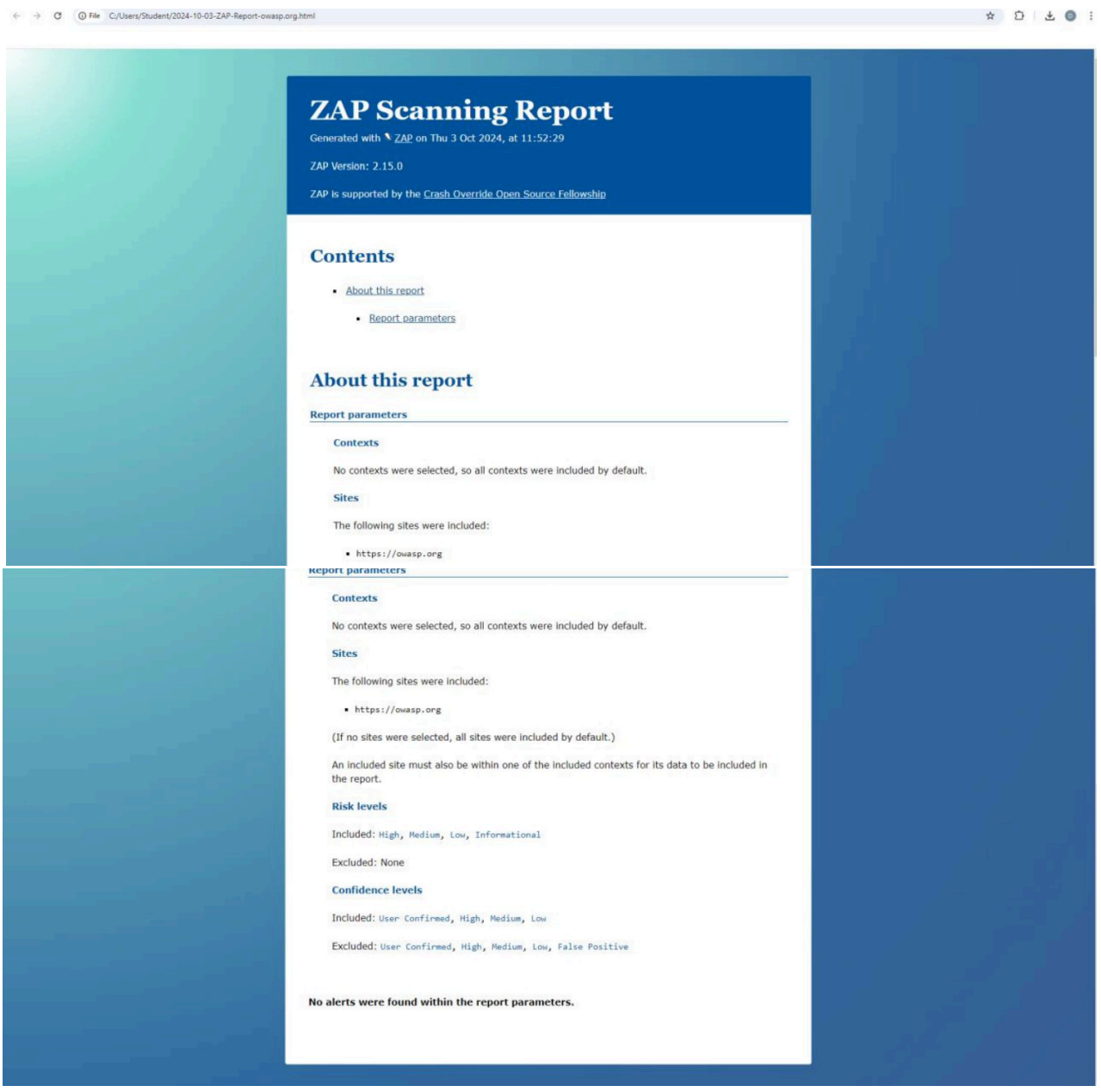


2. Testing









3. Analysis

A. Quality of Web Pages:

- **No Alerts**: This indicates that the website did not show any known vulnerabilities during the scan. This is a positive outcome, implying that the website has passed the security test for common vulnerabilities like Cross-Site Scripting (XSS), SQL Injection, and other attack vectors.
- **Included Risk Levels**: The scan included all risk levels (High, Medium, Low, and Informational), meaning the test was thorough and covered a wide range

of potential security issues.

B. Performance of Security Testing:

- Since no alerts were found, the performance of the website from a security perspective seems satisfactory. However, this does not necessarily mean that the site is completely secure, as ZAP mainly focuses on known vulnerabilities and attack patterns.
- If there are advanced, targeted attacks or undiscovered vulnerabilities, they might not have been identified by this scan.

4. Improvements based on analysis

A. Revalidate with Penetration Testing:

- Since no alerts were found, it is worth conducting manual penetration testing to simulate more targeted attacks. Automated tools can sometimes miss subtle issues that only arise under certain conditions.

B. Regular Audits and Updates:

- Continue regular security audits and ensure that the website software is updated to the latest security patches. Even if no issues are found now, new vulnerabilities are constantly being discovered.

C. Check for Business Logic Flaws:

- While ZAP focuses on technical vulnerabilities, it may not catch business logic flaws—errors that occur in the workflow of an application. It is important to complement this with reviews focused on the application's logic and user workflows.

D. Utilize Additional Tools:

- Consider using other tools alongside ZAP, such as Burp Suite or Arachni, for more comprehensive testing, as different tools might catch different vulnerabilities.

Questions:

1. **Describe security testing. Explain any five test cases/scenarios for testing security of a web application.**

   Ans: Security Testing is a type of software testing intended to uncover vulnerabilities, threats, and risks in a software application. The primary goal is to identify and fix security weaknesses before a malicious attacker can exploit them, thus preventing data loss, unauthorized access, and damage to the organization's reputation. This can be done through dynamic analysis of a running application (DAST) or static analysis of the source code (SAST).

   Here are five test scenarios from the perspective of a Static Analysis (SAST) tool like SonarQube:

   1. **Scan for SQL Injection Vulnerabilities in Code:**
   - Scenario: SonarQube analyzes the source code to find where the application constructs database queries.
   - Goal: It flags any instance where user input is directly concatenated into a SQL query string (e.g., String query = "SELECT * FROM users WHERE name = '" + userName + "';"). This is a classic SQL injection vulnerability, and the tool will recommend using parameterized queries (Prepared Statements) instead.

   2. **Detect Hardcoded Secrets:**
   - Scenario: The scanner reads through all source and configuration files.
   - Goal: To identify sensitive information, such as passwords, API keys, or private certificates, that have been written directly into the code (e.g., String password = "MySuperSecretPassword123!";). This is a major security risk, and the tool will flag it as a critical vulnerability.

   3. **Identify Cross-Site Scripting (XSS) Flaws:**
   - Scenario: The tool traces the flow of data from user input (like a form field) to the final HTML output.
   - Goal: To find code paths where user-controlled data is rendered on a web page without proper escaping or sanitization. This prevents an attacker from injecting malicious scripts that would run in other users' browsers.

   4. **Find Use of Components with Known Vulnerabilities:**
   - Scenario: SonarQube (often with an integrated dependency checker) scans the project's dependency files (e.g., pom.xml, package.json).
   - Goal: To check the versions of all third-party libraries against a database of known vulnerabilities (CVEs). If the project uses a library with a known security flaw (like an old version of Log4j), the tool will raise an alert and recommend updating it.

   5. **Check for Weak Cryptography:**
   - Scenario: The scanner looks for cryptographic functions being used within the code.

- Goal: To identify the use of outdated and insecure hashing algorithms (like MD5 or SHA1) or weak encryption ciphers. It will recommend replacing them with modern, strong alternatives (like SHA-256 and AES-256).

2. **What are the benefits of Security testing for website owners?**
   Ans: Security testing provides numerous crucial benefits for website owners:
   - **Protects Sensitive Data:** It helps find and fix flaws that could lead to data breaches, protecting confidential customer information.
   - **Maintains Customer Trust and Reputation:** A secure website builds trust. A security incident can severely damage a brand's reputation, causing customers to leave.
   - **Prevents Financial Loss:** The costs associated with a breach—including regulatory fines, legal fees, and recovery expenses—can be devastating. Proactive testing is a cost-effective way to prevent this.
   - **Reduces Cost of Fixing Flaws (Shift Left Security):** Using tools like SonarQube to find issues in the source code is significantly cheaper and faster than finding them in a live application. Fixing a bug in development costs a fraction of what it costs to fix it in production.
   - **Aids in Regulatory Compliance:** Many industries have strict compliance standards (like PCI DSS or HIPAA) that mandate secure coding practices and assessments. Static analysis provides documented proof of these efforts.

Outcomes: CO3 – Apply recent automation tools for testing software.

Conclusion: (Conclusion to be based on outcomes)

This experiment involved performing security testing by conducting a static code analysis on a sample project's source code using SonarQube. The SonarScanner was used to analyze the codebase, and the results were published to the SonarQube dashboard for review. The analysis successfully identified a range of issues, including potential security vulnerabilities (like hardcoded credentials or code susceptible to injection), bugs, and code smells. The dashboard provided a clear overview of the project's health and highlighted specific files and line numbers requiring attention, along with detailed guidance for remediation. This exercise demonstrates the value of Static Application Security Testing (SAST) in the "Shift Left" approach to security. By identifying flaws directly in the source code before deployment, SonarQube enables developers to fix issues early, reducing the cost of remediation and preventing vulnerabilities from ever reaching a live production environment.

Grade: AA / AB / BB / BC / CC / CD / DD

Signature of faculty in-charge with date

References:

Books/ Journals/ Websites:

1. ZAP (zaproxy.org)
2. Edge-security group - Wfuzz
3. Wapiti : a Free and Open-Source web-application vulnerability scanner in Python (wapiti-scanner.github.io)
4. w3af - Open Source Web Application Security Scanner
5. sqlmap: automatic SQL injection and database takeover tool
6. Code Quality, Security & Static Analysis Tool with SonarQube | Sonar (sonarsource.com)
7. Google Online Security Blog: Introducing nogotofail—a network traffic security testing tool (googleblog.com)
8. GitHub - amoldp/Grabber-Security-and-Vulnerability-Analysis-
9. Arachni - Web Application Security Scanner Framework – Ecsypno